Input Form (IF)

Score: 2/5 — Significant gaps | Confidence: high

Benchmark: *TOXIGEN* | ***Context:*** *South Asian Safety Researchers — TOXIGEN Cross-Regional Validity Assessment*

Input Form definition: Whether the input signal encoding (text, audio, image parameters) matches deployment conditions.

> Each section below is followed by an evidence table. Three types of evidence are cited: **[QN]** — verbatim quotes extracted from the benchmark paper; **[WEB-N]** — web sources consulted during assessment; **[DN]** / **[DATASET-DN]** — sampled datapoints from the benchmark dataset, with an interpretation note.

Justification

TOXIGEN's text-only modality [Q4, Q27] aligns with the deployment's text-only needs, and the Likert-scale-friendly short-sentence format (≤30 tokens [Q44]) is procedurally compatible. However, the deployment's primary input form requirement is code-mixed Romanized scripts (Hinglish, Romanized Urdu, Tanglish) plus Devanagari/Nastaliq/Bengali/Tamil/Sinhala scripts — none represented in TOXIGEN. Standard NLP tokenizers fail on code-mixed text [WEB-1, WEB-3], creating a signal-distribution mismatch that TOXIGEN's clean standard English pipeline cannot bridge. Dataset profiling also reveals generation artifacts (truncated text, apostrophe corruption, prompt-injection artifacts) that further reduce form reliability.

ID	Evidence
Q4	“We develop a demonstration-based prompting framework and an adversarial classifier-in-the-loop decoding method to generate subtly toxic and benign text with a massive pretrained language model (Brown et al., 2020).” (p.1)
Q27	“TOXIGEN is generated by prompting a language model to produce both benign and toxic sentences that (1) include mentions of minority groups by name and (2) contain mainly implicit language, which does not include profanity or slurs.” (p.3)
Q44	“Overall, by repeating this process for both toxic and benign examples for all 13 target groups, we create 26 sets of prompts. We generate TOXIGEN data with and without ALICE. Without ALICE, we use top-k decoding (Fan et al., 2018) alone with our toxic and benign prompts. With ALICE, we use the HateBERT fine-tuned OffensEval model from Caselli et al. (2021) as the toxicity classifier (CLF). This model covers a range of direct and veiled offense types. We use GPT-3 for the language model. For decoding, we use λL = λC = 0.5, a maximum generation length of 30 tokens, a beam size of 10, and a temperature of 0.9.” (p.4)
WEB-1	Standard NLP tools trained on monolingual data fail on Hinglish code-mixed data (https://link.springer.com/article/10.1007/s13278-022-00920-w)
WEB-3	In Romanized Hindi, a single word may have multiple spelling variants (e.g., 'Tu pagal hai' → 'Tu pgl h'), defeating tokenizers (https://dl.acm.org/doi/10.1145/3748326)

Strengths

- Text-only modality matches the deployment's text-only needs [Q4, Q27].
- Short-sentence ≤30-token format [Q44] is compatible with stimulus-presentation workflows used in toxicity-rating studies.
- Structured release format with explicit fields [Q122] supports programmatic adaptation.

ID	Evidence
Q4	“We develop a demonstration-based prompting framework and an adversarial classifier-in-the-loop decoding method to generate subtly toxic and benign text with a massive pretrained language model (Brown et al., 2020).” (p.1)
Q27	“TOXIGEN is generated by prompting a language model to produce both benign and toxic sentences that (1) include mentions of minority groups by name and (2) contain mainly implicit language, which does not include profanity or slurs.” (p.3)
Q44	“Overall, by repeating this process for both toxic and benign examples for all 13 target groups, we create 26 sets of prompts. We generate TOXIGEN data with and without ALICE. Without ALICE, we use top-k decoding (Fan et al., 2018) alone with our toxic and benign prompts. With ALICE, we use the HateBERT fine-tuned OffensEval model from Caselli et al. (2021) as the toxicity classifier (CLF). This model covers a range of direct and veiled offense types. We use GPT-3 for the language model. For decoding, we use λL = λC = 0.5, a maximum generation length of 30 tokens, a beam size of 10, and a temperature of 0.9.” (p.4)

Q122	“We release TOXIGEN as a dataframe with the following fields: prompt contains the prompts we use for each generation. generation is the TOXIGEN generated text. generation method denotes whether or not ALICE was used to generate the corresponding generation. If this value is ALICE, then ALICE was used, if it is top-k, then ALICE was not used. prompt_label is the binary value indicating whether or not the prompt is toxic (1 is toxic, 0 is benign), and therefore the generation should be toxic as well. This label is slightly noisy, though largely accurate—as deemed by human annotators. group indicates for which group the prompt was generated. Finally, roberta_prediction is the probability predicted by our corresponding RoBERTa model for each instance. This field can be used as propagated labels according to this model.” (p.16)

Checklist

Checklist item definitions:

ID	Assessment Question
IF-1	Compare signal distributions between source and target
IF-2	Check if regional infrastructure supports data capture specs
IF-3	Identify domain-specific form differences
IF-4	Document form mismatches harming external validity

Checklist responses:

IF-1: Signal distribution mismatch: TOXIGEN produces clean standard American English [Q44, Q49]; the deployment requires code-mixed Romanized scripts and multiple non-Latin scripts (Devanagari, Nastaliq, Bengali, Tamil, Sinhala) which differ in tokenization, vocabulary, spelling normalization, and orthography [WEB-1, WEB-3].

IF-2: Regional infrastructure can capture text in all required scripts (this is a research-facing deployment), but standard NLP pipelines and tokenizers do not handle code-mixed Romanized text reliably [WEB-1, WEB-3]. TOXIGEN's pipeline assumptions (standard English tokenization, GPT-3 vocabulary) do not transfer.

IF-3: Domain-specific form differences: code-mixed Romanized text has multiple spelling variants per word [WEB-3], non-standard syntax [WEB-1], and mixed-script content. None of these is present in TOXIGEN's input form.

IF-4: External validity is harmed: classifiers or LLMs evaluated on TOXIGEN's standard English form will not provide informative signal about behavior on code-mixed South Asian inputs.

ID	Evidence
Q4	“We develop a demonstration-based prompting framework and an adversarial classifier-in-the-loop decoding method to generate subtly toxic and benign text with a massive pretrained language model (Brown et al., 2020).” (p.1)
Q27	“TOXIGEN is generated by prompting a language model to produce both benign and toxic sentences that (1) include mentions of minority groups by name and (2) contain mainly implicit language, which does not include profanity or slurs.” (p.3)
Q44	“Overall, by repeating this process for both toxic and benign examples for all 13 target groups, we create 26 sets of prompts. We generate TOXIGEN data with and without ALICE. Without ALICE, we use top-k decoding (Fan et al., 2018) alone with our toxic and benign prompts. With ALICE, we use the HateBERT fine-tuned OffensEval model from Caselli et al. (2021) as the toxicity classifier (CLF). This model covers a range of direct and veiled offense types. We use GPT-3 for the language model. For decoding, we use $\lambda L = \lambda C = 0.5$, a maximum generation length of 30 tokens, a beam size of 10, and a temperature of 0.9.” (p.4)
Q49	“In our final dataset, generation length varies significantly and, as expected, almost all the statements are implicit.” (p.6)
WEB-1	Standard NLP tools trained on monolingual data fail on Hinglish code-mixed data (https://link.springer.com/article/10.1007/s13278-022-00920-w)
WEB-3	In Romanized Hindi, a single word may have multiple spelling variants (e.g., 'Tu pagal hai' → 'Tu pgl h'), defeating tokenizers (https://dl.acm.org/doi/10.1145/3748326)

Information Gaps

- Quantitative measurement of generation-artifact rate across the full 274k corpus is not documented; dataset profiling sample is too small for a reliable estimate.

Remediation

Gap 1: Standard English form does not reflect code-mixed Romanized signal distribution.

Recommendation: Build a stimulus pipeline supporting Romanized code-mixed input with explicit handling of spelling variation [WEB-3] and code-mixed tokenization [WEB-1]. Validate annotation interfaces for Devanagari, Nastaliq, Bengali, Tamil, and Sinhala scripts, plus RTL rendering for Urdu.

ID	Evidence
WEB-1	Standard NLP tools trained on monolingual data fail on Hinglish code-mixed data (https://link.springer.com/article/10.1007/s13278-022-00920-w)
WEB-3	In Romanized Hindi, a single word may have multiple spelling variants (e.g., 'Tu pagal hai' → 'Tu pgl h'), defeating tokenizers (https://dl.acm.org/doi/10.1145/3748326)